Filière : Transformation Digital Industrielle

Module : Génie Logiciel

TD. POO

Réalisée par : Nawal El Aourfe

Encadrant : Pr. BE.ELBAGHAZAOUI

ENSA Beni Mellal — Année universitaire 2025/2026

Exercice 1.1 — Système de gestion d'animaux dans un Zoo

Ce premier exercice modélise un système de gestion d'animaux en s'appuyant sur les quatre piliers de la POO : encapsulation, héritage, abstraction et polymorphisme

Partie 1 : Définitions théoriques

Q1. Encapsulation — Définition et utilisation dans la classe Animal

L'encapsulation consiste à regrouper les données (attributs) et les comportements (méthodes) d'un objet dans une même unité (la classe), tout en contrôlant l'accès aux données via des modificateurs de visibilité.

Dans la classe Animal, l'attribut nom est déclaré private, ce qui empêche tout accès direct depuis l'extérieur de la classe. L'accès se fait uniquement via le getter public getNom() :

private String nom; // accès interdit depuis l'extérieur

public String getNom() { return nom; } // accès contrôlé

Cela protège l'intégrité de la donnée : personne ne peut modifier nom directement sans passer par une méthode prévue à cet effet.

Q2. Héritage — Définition et identification dans le code

L'héritage permet à une classe (sous-classe) de réutiliser les attributs et méthodes d'une autre classe (super-classe), en y ajoutant ou redéfinissant des comportements propres.

Dans ce code, deux classes utilisent l'héritage :

- Mammifere extends Animal : la classe Mammifere hérite de Animal et implémente la méthode abstraite faireDuBruit() en affichant "<nom> grogne."
- Oiseau extends Animal : la classe Oiseau hérite aussi de Animal, implémente faireDuBruit() avec "<nom> chante.", et ajoute la méthode spécifique voler().

Le mot-clé extends matérialise le lien d'héritage. Le mot-clé super() dans le constructeur de chaque sous-classe appelle le constructeur de la classe parente Animal

Q3. Abstraction — Définition et mise en œuvre dans l'exercice

L'abstraction consiste à définir un modèle général (classe abstraite ou interface) qui impose un contrat aux classes dérivées, sans fournir d'implémentation complète. Elle permet de travailler sur des concepts généraux en cachant les détails d'implémentation.

Dans ce code, l'abstraction est mise en œuvre de deux façons :

- La classe Animal est déclarée abstract : elle ne peut pas être instanciée directement (impossible de faire new Animal(...)).
- La méthode faireDuBruit() est déclarée abstract dans Animal : chaque sous-classe concrète (Mammifere, Oiseau) est obligée de fournir sa propre implémentation.

```
public abstract void faireDuBruit();  // contrat imposé aux sous-classes
```

Q4. Polymorphisme — Définition et démonstration dans main()

Le polymorphisme (du grec "plusieurs formes") permet à un même appel de méthode de produire des comportements différents selon le type réel de l'objet manipulé. En Java, c'est le type à l'exécution (et non à la compilation) qui détermine quelle méthode est appelée.

Dans la méthode main(), les deux objets sont déclarés de type Animal mais instanciés en tant que sous-classes concrètes :

Animal tigre = new Mammifere("Tigre");

Animal perroquet = new Oiseau("Perroquet");

tigre.faireDuBruit(); // → "Tigre grogne." (Mammifere)

perroquet.faireDuBruit(); // → "Perroquet chante." (Oiseau)

C'est le même appel de méthode faireDuBruit(), mais le comportement est différent selon l'instance : c'est le polymorphisme par surcharge dynamique (late binding).

Partie 2 : Compléter le codeÉtape

Q5. Méthode voler() dans la classe Oiseau

La méthode voler() a été ajoutée dans la classe Oiseau. Elle affiche un message indiquant que l'oiseau est en vol

```java
// 3. Classe Oiseau – Héritage de Animal
class Oiseau extends Animal {
    public Oiseau(String nom) {
        super(nom);
    }

    @Override
    public void faireDuBruit() {
        System.out.println(getNom() + " chante.");
    }

    // Méthode spécifique aux oiseaux
    public void voler() {
        System.out.println("L'oiseau vole.");
    }
}
```

Q5. Méthode voler() dans la classe Oiseau

La méthode voler() a été ajoutée dans la classe Oiseau. Elle affiche un message indiquant que l'oiseau est en vol

```java
// 4. Classe principale
public class Zoo {
    public static void main(String[] args) {
        Animal tigre     = new Mammifere("Tigre");
        Animal perroquet = new Oiseau("Perroquet");

        // Polymorphisme : même méthode, comportements différents
        tigre.faireDuBruit();      // Tigre grogne.
        perroquet.faireDuBruit();  // Perroquet chante.

        // Utilisation de voler() -> cast nécessaire
        Oiseau o = (Oiseau) perroquet;
        o.voler(); // L'oiseau vole.
    }
}
```

```
<terminated> Zoo [Java Application] C:\Users\HP\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.fu
Tigre grogne.
Perroquet chante.
L'oiseau vole.
```

Partie 3 : Identification des mots-clés

Q7. Mots-clés identifiés dans le code Java

- Héritage : extends → class Mammifere extends Animal
- Redéfinition de méthode : @Override → indique que la méthode réécrit celle de la super-classe
- Abstraction : abstract → abstract class Animal / public abstract void faireDuBruit()
- Appel constructeur parent : super(nom) → dans Mammifere et Oiseau
- Modificateur d'accès : private → attribut nom de Animal

Exercice 1.2 — Système de gestion des véhicules

Cet exercice modélise un système de gestion de véhicules pour une société de transport. Les classes Voiture, Camion et Moto héritent toutes de la classe abstraite Vehicule

Partie 1 : Théorie et concepts

Q1. Encapsulation — Identification dans le code

L'encapsulation est utilisée dans la classe Vehicule pour protéger ses trois attributs principaux, tous déclarés private :

```
private String marque;
private String modele;
private int    annee;
```

L'accès à ces attributs est contrôlé exclusivement via des getters publics : getMarque(), getModele(), getAnnee(). Aucune modification directe n'est possible depuis l'extérieur de la classe, garantissant l'intégrité des données du véhicule.

Q2. Héritage — Implémentation entre Vehicule, Voiture et Camion

La classe abstraite Vehicule constitue la super-classe. Voiture et Camion héritent de Vehicule via le mot-clé extends :

```
class Voiture extends Vehicule { ... }
class Camion  extends Vehicule { ... }
```

Chaque sous-classe appelle super(marque, modele, annee) dans son constructeur pour initialiser les attributs hérités, puis ajoute son propre attribut spécifique :

- Voiture ajoute nombreDePortes (int)
- Camion ajoute capaciteDeCharge (double)
- Moto ajoute typeDeGuidon (String) — voir Q6

Chacune redéfinit (@Override) la méthode abstraite afficherDetails() pour afficher ses informations propres.

Q3. Polymorphisme — Utilisation dans main()

Dans main(), les objets sont déclarés de type Vehicule (type parent) mais instanciés en tant que Voiture, Camion ou Moto :

```
Vehicule maVoiture = new Voiture("Toyota", "Corolla", 2021, 4);
Vehicule monCamion = new Camion("Volvo", "FMX", 2019, 12.5);
Vehicule maMoto    = new Moto("Yamaha", "MT-07", 2022, "Sport");
```

L'appel maVoiture.afficherDetails() déclenche la méthode de Voiture, monCamion.afficherDetails() celle de Camion, etc. — c'est le polymorphisme : même interface, comportements différents selon l'instance réelle.

Q4. Abstraction — Pourquoi Vehicule est-elle abstraite ?

La classe Vehicule est déclarée abstract car un "véhicule générique" n'a pas de sens concret à instancier. On n'instancie jamais directement un Vehicule, seulement des types précis (Voiture, Camion, Moto).

```
public abstract void afficherDetails();  // contrat imposé
```

La méthode abstraite afficherDetails() impose un contrat : toute sous-classe concrète doit obligatoirement définir comment elle affiche ses propres détails. Cela garantit une interface cohérente tout en autorisant des comportements différents.

Partie 2 : Compléter le code

Q5. Méthode demarrer() concrète dans Vehicule + appel dans les sous-classes

La méthode demarrer() a été ajoutée dans Vehicule comme méthode concrète (non abstraite), puis appelée au début de chaque afficherDetails() :

```java
// Méthode concrète commune à tous les véhicules
public void demarrer() {
    System.out.println("Le véhicule démarre.");
}
```

```java
class Voiture extends Vehicule {
    private int nombreDePortes;

    public Voiture(String marque, String modele, int annee, int nombreDePortes) {
        super(marque, modele, annee); // appel constructeur parent
        this.nombreDePortes = nombreDePortes;
    }

    @Override
    public void afficherDetails() {
        demarrer();
        System.out.println("Voiture: " + getMarque() + " " + getModele()
            + " (" + getAnnee() + "), Portes: " + nombreDePortes);
    }
}

// 3. Classe Camion — Héritage de Vehicule
class Camion extends Vehicule {
    private double capaciteDeCharge;

    public Camion(String marque, String modele, int annee, double capaciteDeCharge) {
        super(marque, modele, annee);
        this.capaciteDeCharge = capaciteDeCharge;
    }

    @Override
    public void afficherDetails() {
        demarrer();
        System.out.println("Camion: " + getMarque() + " " + getModele()
            + " (" + getAnnee() + "), Capacité: " + capaciteDeCharge + " tonnes");
```

Q6. Nouvelle classe Moto héritant de Vehicule

La classe Moto a été créée avec un attribut spécifique typeDeGuidon et l'implémentation complète de afficherDetails() :

```java
// 4. Classe Moto — Nouvelle classe héritant de Vehicule
class Moto extends Vehicule {
    private String typeDeGuidon;

    public Moto(String marque, String modele, int annee, String typeDeGuidon) {
        super(marque, modele, annee);
        this.typeDeGuidon = typeDeGuidon;
    }

    @Override
    public void afficherDetails() {
        demarrer();
        System.out.println("Moto: " + getMarque() + " " + getModele()
            + " (" + getAnnee() + "), Guidon: " + typeDeGuidon);
    }
}
```

Interprétation : La classe Moto respecte exactement le même patron que Voiture et Camion : appel de super() pour les attributs communs, attribut propre typeDeGuidon, et redéfinition de afficherDetails() avec appel à demarrer(). Cela illustre la puissance de l'héritage : le code commun ne se répète pas

Partie 3 : Identification des mots-clés

Q7. Mots-clés identifiés dans le code Java

- @Override → redéfinition de méthode (afficherDetails() dans Voiture, Camion, Moto)
- extends → héritage (class Voiture extends Vehicule)
- super(...) → appel au constructeur parent depuis chaque sous-classe
- abstract → classe Vehicule et méthode afficherDetails()
- private → attributs encapsulés dans Vehicule et dans chaque sous-classe

Exercice 1.3 — Gestion des comptes bancaires

Cet exercice implémente un système de comptes bancaires en Java. Il s'agit d'une implémentation complète (et non d'un code partiellement fourni), mettant en pratique l'encapsulation, l'héritage et la spécialisation de classes

```java

// 1. Classe de base CompteBancaire
class CompteBancaire {
    private String numeroCompte; // Encapsulation : privé
    protected double solde;      // protected : accessible aux sous-classes

    public CompteBancaire(String numeroCompte, double soldeInitial) {
        this.numeroCompte = numeroCompte;
        this.solde        = soldeInitial;
    }

    public String getNumeroCompte() { return numeroCompte; }
    public double getSolde()        { return solde; }

    public void deposer(double montant) {
        if (montant > 0) {
            solde += montant;
            System.out.println("Dépôt de " + montant + "€. Solde: " + solde + "€");
        } else {
            System.out.println("Montant invalide.");
        }
    }

    public void retirer(double montant) {
        if (montant > solde) {
            System.out.println("Solde insuffisant ! Solde actuel: " + solde + "€");
        } else {
            solde -= montant;
            System.out.println("Retrait de " + montant + "€. Solde: " + solde + "€");
        }
    }
}
// 2. Classe CompteCourant — Héritage de CompteBancaire
class CompteCourant extends CompteBancaire {
    public CompteCourant(String numeroCompte, double soldeInitial) {
        super(numeroCompte, soldeInitial);
    }
}

// 3. Classe CompteEpargne — Héritage + méthode supplémentaire
class CompteEpargne extends CompteBancaire {
    public CompteEpargne(String numeroCompte, double soldeInitial) {
        super(numeroCompte, soldeInitial);
    }

    // Méthode spécifique : calcul des intérêts
    public void calculerInterets(double taux) {
        double interets = solde * taux / 100;
        solde += interets;
        System.out.println("Intérêts (" + taux + "%): +" + interets + "€. Solde: " + solde + "€");
    }
}

// 4. Classe principale
public class Banque {
    public static void main(String[] args) {
        CompteCourant  cc = new CompteCourant("CC-001", 1000.0);
        CompteEpargne  ce = new CompteEpargne("CE-001", 5000.0);

        System.out.println("=== Compte Courant (" + cc.getNumeroCompte() + ") ===");
        cc.deposer(500);
        cc.retirer(200);
        cc.retirer(2000); // test : solde insuffisant

        System.out.println("\n=== Compte Épargne (" + ce.getNumeroCompte() + ") ===");
        ce.deposer(1000);
        ce.calculerInterets(3.5); // 3.5% d'intérêts sur 6000€
    }
}
```

```
<terminated> Banque [Java Application] C:\Users\HP\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot
Intérêts de 100.0 € ajoutés. Nouveau solde : 2100.0 €
Retrait de 500.0 € effectué. Nouveau solde : 1600.0 €
Compte CE456 - Solde : 1600.0 €
```

Le programme met en œuvre une modélisation orientée objet cohérente d'un système bancaire en s'appuyant sur plusieurs principes fondamentaux. La classe de base *CompteBancaire* applique une encapsulation stricte en déclarant l'attribut *numeroCompte* comme privé, garantissant ainsi qu'il n'est accessible qu'à l'intérieur de la classe. En revanche, l'attribut *solde* est déclaré *protected*, ce qui permet aux classes dérivées d'y accéder directement tout en conservant un certain niveau de contrôle. Les méthodes *deposer()* et *retirer()* jouent un rôle central en assurant la validation des opérations : elles empêchent respectivement les dépôts négatifs et les retraits dépassant le solde disponible, ce qui renforce la robustesse du système.

Dans cette architecture, *CompteCourant* représente une spécialisation minimale de *CompteBancaire*. Elle illustre clairement le principe de réutilisation par héritage, puisqu'elle hérite directement des méthodes existantes sans les redéfinir, évitant ainsi toute duplication de code. À l'inverse, *CompteEpargne* enrichit la classe parente en introduisant une fonctionnalité spécifique via la méthode *calculerInterets()*. Cette méthode exploite directement l'attribut *solde* grâce à sa visibilité *protected* et applique une formule simple (intérêts = solde × taux / 100) avant d'ajouter les intérêts au solde.

Globalement, ce programme démontre une utilisation structurée des concepts de la programmation orientée objet : l'encapsulation pour sécuriser les données, l'héritage pour factoriser et réutiliser le code, et la spécialisation pour adapter ou étendre le comportement des classes selon des besoins métiers spécifiques.